

Deep Learning

Single Layer Perceptron বনাম Multi-Layer Perceptron

একটি $3 \rightarrow 2 \rightarrow 1$ নেটওয়ার্কের সম্পূর্ণ *Forward Propagation*

Md. Shaon Khan

B.Sc. in Information Technology, Institute of Information Technology (IIT), Jahangirnagar University

1. ভূমিকা (Introduction)

Neural Network শেখার শুরুতে সবচেয়ে গুরুত্বপূর্ণ প্রশ্ন হলো — একটি Network কীভাবে Input থেকে একটি Prediction তৈরি করে? এই প্রশ্নের উত্তর খুঁজতে গেলে প্রথমেই আসে দুটি ধারণা: **Single Layer Perceptron (SLP)** এবং **Multi-Layer Perceptron (MLP)**। এই নোটে আমরা দেখব SLP আসলে কী, কেন সেটা যথেষ্ট নয়, MLP কীভাবে এই সীমাবদ্ধতা দূর করে, এবং একটি ছোট $3 \rightarrow 2 \rightarrow 1$ নেটওয়ার্কে ধাপে ধাপে **Forward Propagation** কীভাবে কাজ করে — হাতে-কলমে সংখ্যাসহ।

গণিতকে ভয় না পাইয়ে, প্রতিটি Formula একদম ছোট এক-Neuron উদাহরণ থেকে শুরু করে ধীরে ধীরে সম্পূর্ণ Matrix Notation পর্যন্ত নিয়ে যাওয়া হয়েছে। লক্ষ্য তথ্যের পরিমাণ বাড়ানো নয় — লক্ষ্য হলো ধারণাটা সম্পূর্ণভাবে বোঝা।

2. Single Layer Perceptron (SLP)

সংজ্ঞা (Definition)

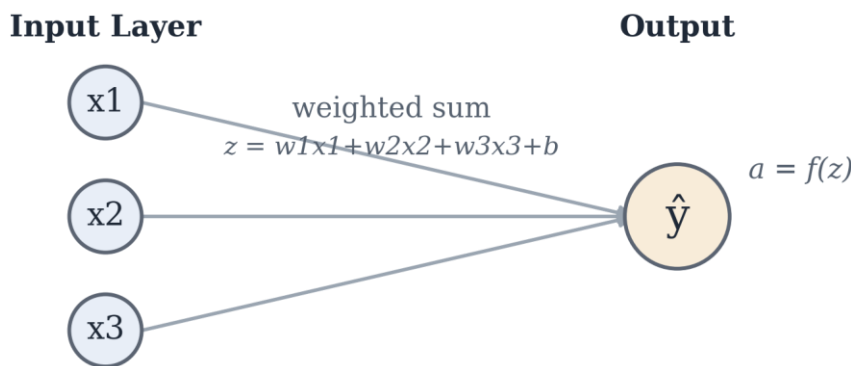
Definition

একটি **Perceptron** হলো Neural Network-এর সবচেয়ে ছোট একক (unit) — এটি কয়েকটি **Input** নেয়, প্রতিটির সাথে একটি **Weight** গুণ করে, সব যোগফলের সাথে একটি **Bias** যোগ করে, এবং শেষে একটি **Activation Function** দিয়ে চূড়ান্ত মান বের করে। যখন কোনো **Hidden Layer** ছাড়াই সরাসরি Input থেকে Output-এ চলে যাওয়া হয়, তখন তাকে বলে **Single Layer Perceptron**।

গঠন (Architecture)

SLP-এর গঠন খুবই সহজ — কোনো মধ্যবর্তী স্তর নেই, Input সরাসরি Output-এর সাথে যুক্ত:

Input → **Weighted Sum** → **Activation** → **Output**



চিত্র 1: Single Layer Perceptron — তিনটি Input সরাসরি একটি Output Neuron-এ যুক্ত।

কীভাবে কাজ করে (Working)

প্রতিটি Input তার নিজস্ব Weight দিয়ে গুণ হয়, সবগুলো গুণফল যোগ হয়, তারপর একটি Bias যোগ হয় — এই মোট মানটাই z । এরপর z -কে একটি Activation Function-এর মধ্য দিয়ে পাঠিয়ে চূড়ান্ত Output a পাওয়া যায়।

গাণিতিক রূপ (Mathematical Representation)

প্রথমে একটি মাত্র Neuron-এর হিসাব দেখা যাক:

$$z = w_1x_1 + w_2x_2 + w_3x_3 + b$$

এখানে —

- x = Input
- w = Weight (প্রতিটি Input কতটা গুরুত্বপূর্ণ তা নির্ধারণ করে)
- b = Bias (একটি অতিরিক্ত স্বাধীন মান, যা Decision Boundary-কে সরাসরি সাহায্য করে)
- z = Weighted Sum + Bias — অর্থাৎ, Activation-এর ঠিক আগের মান

এরপর এই z -এর উপর Activation Function বসিয়ে চূড়ান্ত Output বের করা হয়:

$$a = f(z)$$

এখানে f যেকোনো Activation Function হতে পারে (যেমন Sigmoid); a হলো Activation-এর **পরের মান** — যেটাই মডেলের চূড়ান্ত Prediction হিসেবে ব্যবহৃত হয়।

সীমাবদ্ধতা (Limitation)

Important Note

SLP শুধুমাত্র একটি সরল রেখা (Linear Decision Boundary) দিয়ে দুটি ক্লাসকে আলাদা করতে পারে। ফলে এটি শুধু **Linearly Separable** সমস্যার সমাধান করতে পারে — অর্থাৎ যেখানে একটিমাত্র সরলরেখা দিয়ে দুই ধরনের ফলাফল আলাদা করা যায়। যেখানে ডেটা এভাবে আলাদা করা যায় না (যেমন XOR), সেখানে SLP ব্যর্থ হয়। পরের অংশে আমরা এটি বিস্তারিত দেখব।

3. কেন Single Layer Perceptron যথেষ্ট নয়

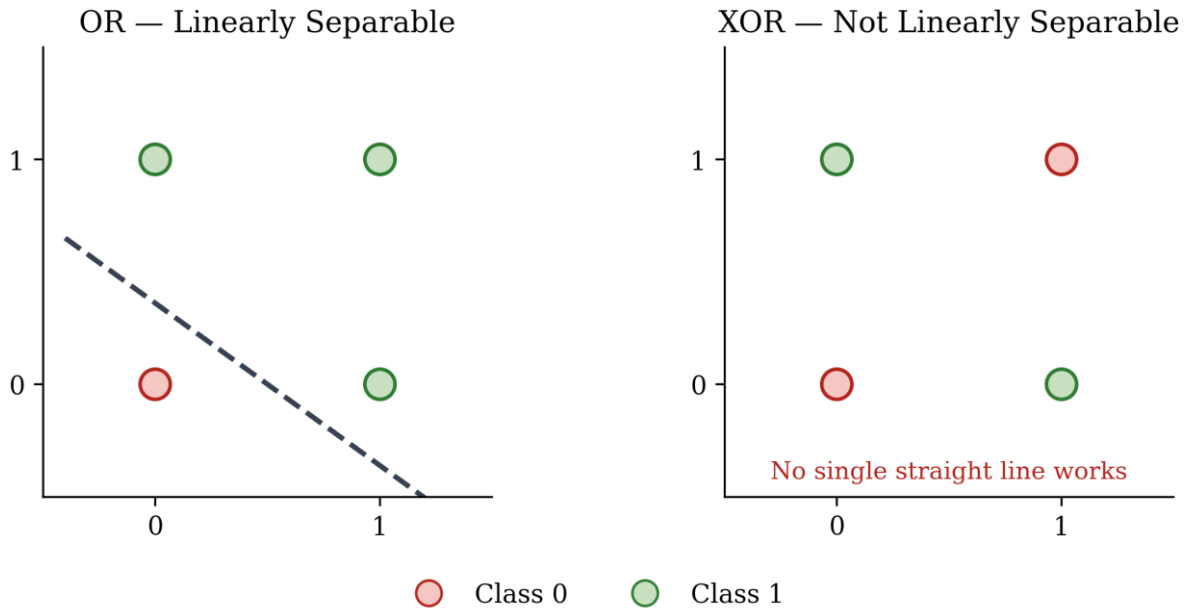
Linear Decision Boundary

একটি Perceptron আসলে একটি সরলরেখা (২ডি-তে) বা একটি Hyperplane (উচ্চ-মাত্রায়) আঁকে, যা দুটি Class-কে আলাদা করার চেষ্টা করে। এই রেখার একপাশে Class 0 এবং অন্যপাশে Class 1 — এটাই এর একমাত্র সিদ্ধান্ত নেওয়ার ক্ষমতা।

Linearly Separable বনাম Non-linearly Separable

- **Linearly Separable:** একটিমাত্র সরলরেখা দিয়ে দুই Class সম্পূর্ণ আলাদা করা যায়। উদাহরণ: AND, OR

- **Non-linearly Separable:** কোনো একটিমাত্র সরলরেখা দিয়ে আলাদা করা সম্ভব নয়। উদাহরণ: XOR



চিত্র 2: OR-এর ক্ষেত্রে একটি সরলরেখা দুই Class আলাদা করতে পারে, কিন্তু XOR-এর ক্ষেত্রে কোনো সরলরেখা কাজ করে না।

কেন XOR-এর জন্য একটিমাত্র সরলরেখা যথেষ্ট নয়

XOR গেটে, $(0,0)$ ও $(1,1)$ একটি Class-এর, এবং $(0,1)$ ও $(1,0)$ অন্য Class-এর। এই চারটি বিন্দু এমনভাবে সাজানো যে তাদের মাঝে একটিমাত্র সরলরেখা টানলে সবসময় দুই Class মিশে যাবে — উপরের চিত্রে এটি স্পষ্ট। তাই একটি একক Linear Decision Boundary দিয়ে XOR সমাধান করা গাণিতিকভাবেই অসম্ভব। এই সীমাবদ্ধতা কাটাতেই দরকার হয় Hidden Layer, অর্থাৎ Multi-Layer Perceptron।

4. Multi-Layer Perceptron (MLP)

সংজ্ঞা (Definition)

Definition

যখন Input এবং Output-এর মাঝে এক বা একাধিক অতিরিক্ত স্তর — অর্থাৎ **Hidden Layer** — যোগ করা হয়, তখন তাকে বলে **Multi-Layer Perceptron (MLP)**। প্রতিটি Hidden Neuron ঠিক সেই একই Weighted Sum + Activation প্রক্রিয়া চালায় যা SLP-তে দেখেছি, শুধু এবার সেটা একাধিক ধাপে ঘটে।

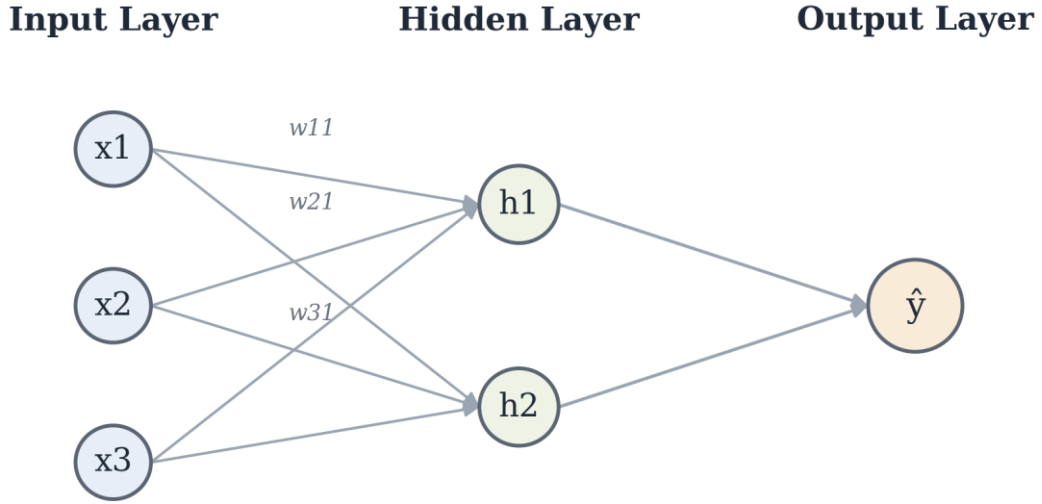
গঠন (Architecture)

Input → Hidden Layer → Output

এই নোটে আমরা পুরোটা জুড়ে একটিমাত্র নেটওয়ার্ক ব্যবহার করব, যাতে ধারণাগুলো সহজে মনে থাকে:

3 → 2 → 1

অর্থাৎ ৩টি Input Neuron, ২টি Hidden Neuron, এবং ১টি Output Neuron।



চিত্র 3: 3 → 2 → 1 MLP আর্কিটেকচার — Input Layer, Hidden Layer এবং Output Layer এর সংযোগ।

Hidden Layer কী

Hidden Layer হলো Input এবং Output-এর মাঝে থাকা Neuron-এর একটি স্তর, যেটা সরাসরি ব্যবহারকারী দেখতে পায় না (তাই নাম Hidden), কিন্তু এখানেই আসল হিসাব-নিকাশ হয়। প্রতিটি Hidden Neuron তার নিজের মতো করে Input-গুলোর একটি Weighted Sum বের করে এবং Activation প্রয়োগ করে একটি নতুন মান তৈরি করে।

কেন Hidden Layer দরকার

Hidden Layer ছাড়া নেটওয়ার্ক শুধু একটি সরলরেখা আঁকতে পারে। কিন্তু Hidden Layer-এর প্রতিটি Neuron আলাদা আলাদা একটি সরলরেখা/Boundary তৈরি করে, এবং Output Layer সেই সবগুলো Boundary-কে Non-linear ভাবে একত্র করে — ফলে নেটওয়ার্ক অনেক জটিল, বাঁকানো Decision Boundary তৈরি করতে পারে, যা দিয়ে XOR-এর মতো সমস্যাও সমাধান করা যায়।

5. SLP বনাম MLP

নিচের টেবিলে দুটি Architecture-এর মূল পার্থক্যগুলো এক নজরে দেখানো হলো:

Feature	Single Layer Perceptron	Multi-Layer Perceptron
Hidden Layer	নেই	আছে
Structure	Input → Output	Input → Hidden → Output
Complexity	কম	বেশি
Linear Problem	পারে	পারে
Non-linear Problem	পারে না	পারে

XOR	পারে না	পারে
Representation Learning	সীমিত	বেশি সক্ষম

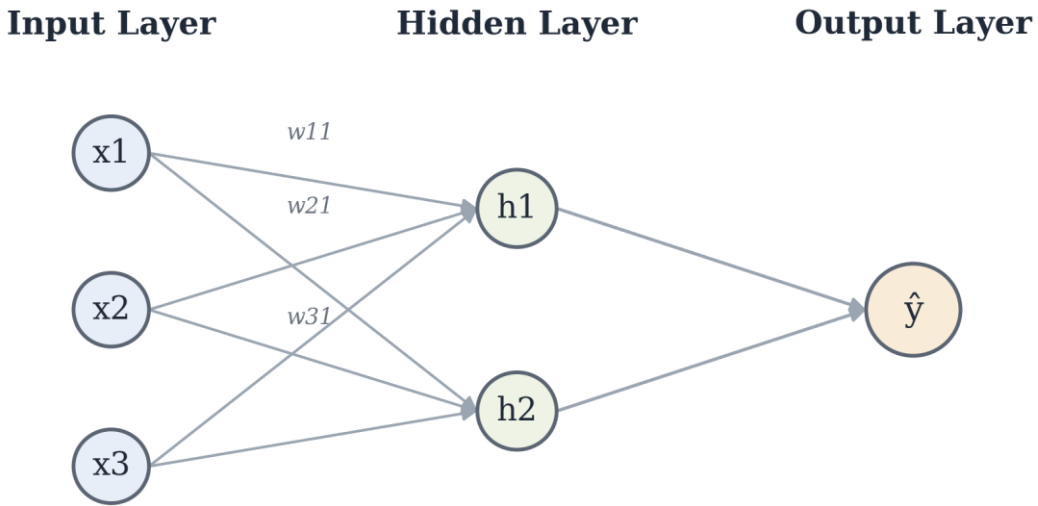
সংক্ষেপে — SLP সহজ কিন্তু সীমিত; MLP কিছুটা জটিল, তবে Hidden Layer-এর কারণে অনেক বেশি ধরনের সমস্যা সমাধান করতে সক্ষম।

6. একটি $3 \rightarrow 2 \rightarrow 1$ MLP বোঝা

এই নোটের বাকি অংশে আমরা একটিমাত্র নির্দিষ্ট নেটওয়ার্ক নিয়ে কাজ করব, যাতে প্রতিটি ধাপ সহজে অনুসরণ করা যায়:

- **Input Layer:** ৩টি Neuron — $x1, x2, x3$
- **Hidden Layer:** ২টি Neuron — $h1, h2$
- **Output Layer:** ১টি Neuron — $\hat{y}$

Input Layer-এর প্রতিটি Neuron, Hidden Layer-এর প্রতিটি Neuron-এর সাথে যুক্ত (একে বলে **Fully Connected**)। একইভাবে Hidden Layer-এর উভয় Neuron যুক্ত থাকে Output Neuron-এর সাথে। নিচের চিত্রটি এই সম্পূর্ণ সংযোগ দেখাচ্ছে — এই একই চিত্র আমরা সামনে সংখ্যাসহ আবার ব্যবহার করব।



চিত্র 4: এই নোটে ব্যবহৃত $3 \rightarrow 2 \rightarrow 1$ নেটওয়ার্ক। প্রতিটি Input, প্রতিটি Hidden Neuron-এর সাথে যুক্ত (উদাহরণস্বরূপ $w11, w21, w31$ দেখানো হলো)।

প্রতিটি সংযোগ (arrow)-এর নিজস্ব একটি Weight আছে, এবং প্রতিটি Neuron-এর নিজস্ব একটি Bias আছে। এই Weight ও Bias-গুলোই Training-এর সময় শেখা (learn) হয় — কিন্তু এই নোটে আমরা শুধু Forward Propagation, অর্থাৎ ধরে নেওয়া Weight দিয়ে Prediction বের করা শিখব।

7. Forward Propagation

Forward Propagation মানে হলো — Input থেকে শুরু করে ধাপে ধাপে হিসাব করে শেষ পর্যন্ত Output-এ পৌঁছানো। নিচে সম্পূর্ণ প্রক্রিয়াটি ৫টি ধাপে ভাগ করে দেখানো হলো।

ধাপ ১ — Input

প্রথম ধাপ হলো Input-গুলো নেওয়া। আমাদের নেটওয়ার্কে ৩টি Input আছে, যেগুলোকে একসাথে একটি Vector X হিসেবে লেখা যায়:

$$X = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

- **কী হিসাব করছি:** Input মানগুলো সংগ্রহ করছি।
- **কেন করছি:** এই মানগুলোই নেটওয়ার্কে ঢোকানোর একমাত্র তথ্য (feature)।
- **পরে কী কাজে লাগবে:** এই X পরের ধাপে প্রতিটি Hidden Neuron-এর হিসাবে ব্যবহৃত হবে।

ধাপ ২ — Input → Hidden Layer (Weighted Sum)

Hidden Layer-এর প্রতিটি Neuron আলাদাভাবে তার নিজস্ব Weighted Sum বের করে। Hidden Neuron 1-এর জন্য:

$$z_1 = w_{11}x_1 + w_{21}x_2 + w_{31}x_3 + b_1$$

এবং Hidden Neuron 2-এর জন্য:

$$z_2 = w_{12}x_1 + w_{22}x_2 + w_{32}x_3 + b_2$$

লক্ষ করুন — দুটি Neuron ঠিক একই ধরনের কাজ করছে, শুধু তাদের নিজস্ব আলাদা Weight (w) ও Bias (b) ব্যবহার করে। z_1 ও z_2 হলো Activation প্রয়োগ করার আগের মান।

- **কী হিসাব করছি:** প্রতিটি Hidden Neuron-এর Weighted Sum + Bias।
- **কেন করছি:** Input-গুলোকে একত্র করে একটি একক সংখ্যায় রূপান্তর করছি, যা পরবর্তী ধাপে Activation-এ যাবে।
- **পরে কী কাজে লাগবে:** এই z মান দুটি Activation Function-এ প্রবেশ করবে।

ধাপ ৩ — Activation প্রয়োগ

এখন প্রতিটি z মানের উপর একটি Non-linear Activation Function বসানো হবে — এখানে উদাহরণ হিসেবে Sigmoid ব্যবহার করা হয়েছে:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

তাহলে দুটি Hidden Neuron-এর Activation হবে:

$$a_1 = \sigma(z_1) \quad a_2 = \sigma(z_2)$$

Important Note

z হলো Activation-এর **আগের** মান, এবং a হলো Activation-এর **পরের** মান। এই দুটি প্রতীক সবসময় আলাদাভাবে মনে রাখা জরুরি।

- **কী হিসাব করছি:** Non-linear Activation প্রয়োগ করছি।
- **কেন করছি:** শুধু Weighted Sum (Linear অপারেশন) দিয়ে নেটওয়ার্ক কখনো Non-linear সমস্যা সমাধান করতে পারবে না — Activation-ই Non-linearity যোগ করে।
- **পরে কী কাজে লাগবে:** a_1 ও a_2 এখন Output Layer-এর Input হিসেবে ব্যবহৃত হবে।

ধাপ ৪ — Hidden → Output

Output Neuron ঠিক একই পদ্ধতিতে কাজ করে — শুধু এবার Input হিসেবে আসল Input-এর বদলে Hidden Layer-এর Output (a_1, a_2) ব্যবহৃত হয়:

$$z^{[2]} = w_1 a_1 + w_2 a_2 + b$$

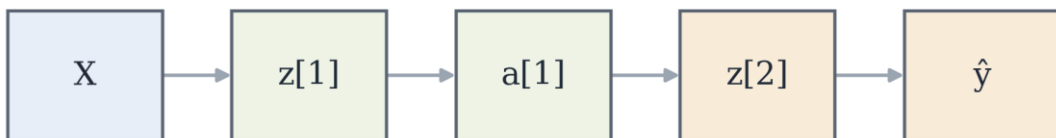
- **কী হিসাব করছি:** Output Layer-এর Weighted Sum।
- **কেন করছি:** Hidden Layer থেকে পাওয়া নতুন Representation-কে একত্র করে একটি একক মানে আনছি।
- **পরে কী কাজে লাগবে:** $z^{[2]}$ এখন চূড়ান্ত Activation-এ যাবে।

ধাপ ৫ — চূড়ান্ত Activation

সবশেষে $z^{[2]}$ -এর উপর Activation প্রয়োগ করলেই মডেলের চূড়ান্ত Prediction পাওয়া যায়:

$$\hat{y} = \sigma(z^{[2]})$$

- **কী হিসাব করছি:** চূড়ান্ত Output $\hat{y}$ (y-hat)।
- **কেন করছি:** এটাই মডেলের চূড়ান্ত Prediction — যেমন একটি Binary Classification-এ ০ থেকে ১-এর মধ্যে একটি সম্ভাবনা।
- **পরে কী কাজে লাগবে:** Training-এর সময় এই $\hat{y}$ -কে প্রকৃত Label-এর সাথে তুলনা করে Loss হিসাব করা হয় (যা এই নোটের আওতার বাইরে)।



8. সম্পূর্ণ সংখ্যাসহ উদাহরণ

এখন উপরের সবগুলো ধাপ প্রকৃত সংখ্যা দিয়ে হিসাব করে দেখা যাক। ধরি, Input:

$$x_1 = 2 \quad x_2 = 3 \quad x_3 = 1$$

Hidden Neuron 1

ধরি $w_{11}=0.5$, $w_{21}=0.2$, $w_{31}=0.1$, $b_1=0.1$ । তাহলে —

$$z_1 = (0.5)(2) + (0.2)(3) + (0.1)(1) + 0.1$$

$$z_1 = 1.8$$

এখন Sigmoid প্রয়োগ করলে:

$$a_1 = \sigma(1.8) \approx 0.858$$

Hidden Neuron 2

ধরি $w_{12}=0.3$, $w_{22}=-0.1$, $w_{32}=0.4$, $b_2=-0.2$ । তাহলে —

$$z_2 = (0.3)(2) + (-0.1)(3) + (0.4)(1) + (-0.2)$$

$$z_2 = 0.5$$

এখন Sigmoid প্রয়োগ করলে:

$$a_2 = \sigma(0.5) \approx 0.622$$

Output Neuron

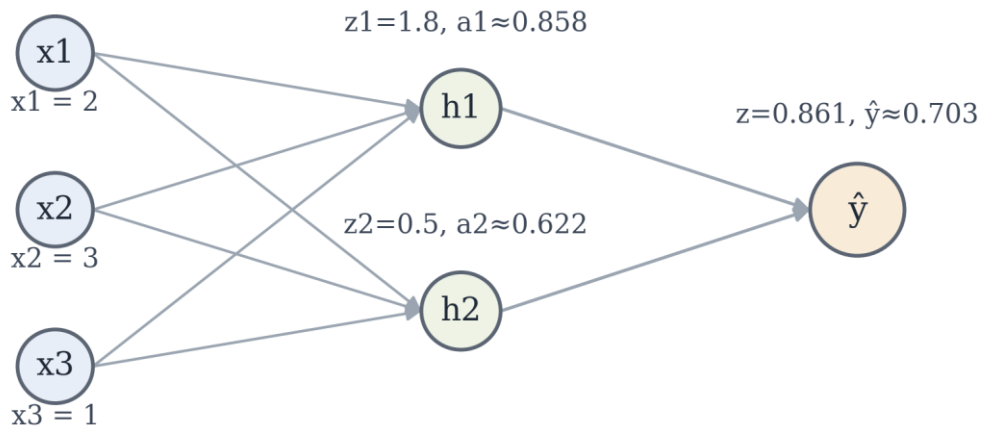
ধরি Output Layer-এর Weight ও Bias — $w_1=0.7$, $w_2=0.9$, $b=-0.3$ । তাহলে —

$$z^{[2]} = (0.7)(0.858) + (0.9)(0.622) + (-0.3)$$

$$z^{[2]} \approx 0.861$$

সবশেষে চূড়ান্ত Prediction:

$$\hat{y} = \sigma(0.861) \approx 0.703$$



চিত্র 6: সংখ্যাসহ সম্পূর্ণ Forward Propagation — Input (2, 3, 1) থেকে $\hat{y} \approx 0.703$ পর্যন্ত।

Key Takeaway

তিনটি Input (2, 3, 1) থেকে শুরু করে, দুটি Hidden Neuron-এর মধ্য দিয়ে, শেষ পর্যন্ত একটি একক সংখ্যা $\hat{y} \approx 0.703$ পাওয়া গেল — এটাই সম্পূর্ণ Forward Propagation-এর ফলাফল। উপরের চিত্রে পুরো হিসাবটা একসাথে দেখানো হলো।

9. ম্যাট্রিক্স আকারে প্রকাশ (Matrix Representation)

উপরে আমরা প্রতিটি Neuron-এর হিসাব আলাদা আলাদাভাবে করেছি। কিন্তু বাস্তবে যখন শত শত বা হাজার হাজার Neuron থাকে, তখন প্রতিটি আলাদাভাবে লেখা অসম্ভব। তাই একই ধরনের অনেকগুলো হিসাব একসাথে কমপ্যাক্টভাবে করার জন্য ব্যবহার করা হয় **Matrix Notation**।

নতুন প্রতীক পরিচিতি

প্রথমে কয়েকটি নতুন প্রতীক পরিচয় করিয়ে দেওয়া যাক:

$$W^{[1]}, b^{[1]}, W^{[2]}, b^{[2]}$$

- $W^{[1]}$ = Input $\rightarrow$ Hidden সংযোগের সবগুলো Weight একসাথে (একটি Matrix)
- $b^{[1]}$ = Hidden Layer-এর সবগুলো Bias একসাথে
- $W^{[2]}$ = Hidden $\rightarrow$ Output সংযোগের সবগুলো Weight একসাথে
- $b^{[2]}$ = Output Layer-এর Bias

Matrix আকারে সম্পূর্ণ Forward Propagation

আগের ধাপ ২ ও ৩-এ আমরা z_1 ও z_2 আলাদা আলাদা করে লিখেছিলাম। Matrix Notation-এ এই দুটি হিসাব একসাথে একটি লাইনে লেখা যায়:

$$z^{[1]} = W^{[1]}X + b^{[1]}$$

$$a^{[1]} = f(z^{[1]})$$

একইভাবে Output Layer-এর জন্য:

$$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}$$

$$\hat{y} = a^{[2]} = f(z^{[2]})$$

এখানে ৪টি লাইনই ঠিক আগের ধাপগুলোরই সংক্ষিপ্ত রূপ — নতুন কোনো ধারণা নয়, শুধু লেখার পদ্ধতি বদলেছে।

Optional: গাণিতিক সম্প্রসারণ (Mathematical Expansion)

যদি সবগুলো ধাপ একসাথে একটি মাত্র Equation-এ লেখা হয়, তাহলে তা দেখতে এমন হবে (এটি মুখস্থ করার দরকার নেই — এটি শুধু উপরের ছোট ছোট ধাপগুলোরই সমষ্টি):

$$\hat{y} = \sigma(w_1 \sigma(w_{11}x_1 + w_{21}x_2 + w_{31}x_3 + b_1) + w_2 \sigma(w_{12}x_1 + w_{22}x_2 + w_{32}x_3 + b_2) + b)$$

10. W, b, z, a — একসাথে পুনরালোচনা

এই পর্যন্ত আমরা বেশ কিছু প্রতীক ব্যবহার করেছি। নিচের টেবিলে সবগুলো একসাথে সংক্ষেপে দেখানো হলো, যাতে সহজে রিভিশন করা যায়:

Symbol	অর্থ (Meaning)
x	Input
w	Weight
b	Bias
z	Weighted Sum + Bias (Activation-এর আগের মান)
a	Activated Output (Activation-এর পরের মান)
f	Activation Function
$W^{[1]}$	প্রথম স্তরের (Input → Hidden) সবগুলো Weight
$W^{[2]}$	দ্বিতীয় স্তরের (Hidden → Output) সবগুলো Weight
$b^{[1]}$	Hidden Layer-এর Bias
$b^{[2]}$	Output Layer-এর Bias

$\hat{y}$	মডেলের চূড়ান্ত Prediction
-----------	----------------------------

সহজভাবে মনে রাখার উপায়: প্রতিটি Neuron সবসময় দুটি কাজ করে — প্রথমে একটি **Linear** কাজ ($z = wx + b$), তারপর একটি **Non-linear** কাজ ($a = f(z)$)। পুরো নেটওয়ার্ক আসলে এই দুই ধাপের বারবার পুনরাবৃত্তি।

11. Dimension Check

Matrix হিসাবের সময় সবচেয়ে গুরুত্বপূর্ণ বিষয় হলো প্রতিটি Matrix-এর আকার (Dimension) ঠিক আছে কিনা তা যাচাই করা। আমাদের নেটওয়ার্কে:

Input → Hidden স্তর

রাশি (Quantity)	আকার (Dimension)
X (Input)	3×1
W[1] (Weight)	2×3
W[1]X	2×1
b[1] (Bias)	2×1
z[1] (ফলাফল)	2×1

লক্ষ করুন: W[1] হলো 2×3 (কারণ ২টি Hidden Neuron, প্রতিটির ৩টি Input-এর Weight আছে), এবং X হলো 3×1 । তাই W[1] ও X-কে সরাসরি গুণ করা যায় (Matrix Multiplication-এর নিয়ম অনুযায়ী (2×3) গুণ $(3 \times 1) = (2 \times 1)$)। ফলে এখানে আলাদা করে কোনো Transpose-এর প্রয়োজন নেই।

Hidden → Output স্তর

রাশি (Quantity)	আকার (Dimension)
a[1] (Hidden Output)	2×1
W[2] (Weight)	1×2
W[2]a[1]	1×1
b[2] (Bias)	1×1
z[2] (ফলাফল)	1×1

একইভাবে W[2] হলো 1×2 (১টি Output Neuron, ২টি Hidden Input), এবং a[1] হলো 2×1 । তাই (1×2) গুণ $(2 \times 1) = (1 \times 1)$ — একটিমাত্র সংখ্যা, যা আমাদের প্রত্যাশিত।

12. Universal Approximation Theorem

MLP এত শক্তিশালী কেন, তার একটি তাত্ত্বিক ভিত্তি দেয় **Universal Approximation Theorem**

Definition

এই Theorem বলে — উপযুক্ত শর্তে, একটি যথেষ্ট চওড়া (Wide) Hidden Layer এবং উপযুক্ত Non-linear Activation Function থাকলে, একটি Neural Network বিস্তৃত পরিসরের Continuous Function-কে অনুমান (Approximate) করতে পারে।

Important Note

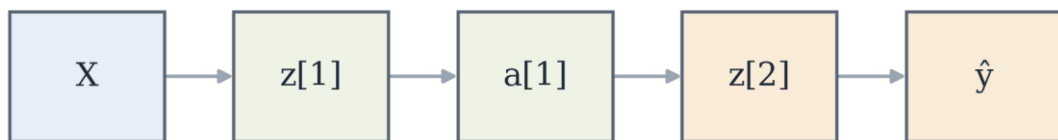
এই Theorem কখনোই বলে না যে — "MLP যেকোনো সমস্যা সমাধান করতে পারে।" এটি শুধু বলে যে তাত্ত্বিকভাবে এমন একটি নেটওয়ার্ক থাকতে পারে যা একটি নির্দিষ্ট Function-কে ভালোভাবে অনুমান করে — বাস্তবে সেই নেটওয়ার্ক খুঁজে বের করা (Training করা), পর্যাপ্ত Data থাকা, এবং সঠিকভাবে Generalize করা সম্পূর্ণ আলাদা একটি চ্যালেঞ্জ।

13. Key Takeaways

- SLP হলো Hidden Layer ছাড়া একটি Neuron, যা শুধু Linearly Separable সমস্যা সমাধান করতে পারে।
- MLP-এ এক বা একাধিক Hidden Layer থাকে, যা নেটওয়ার্ককে Non-linear সমস্যা (যেমন XOR) সমাধানের ক্ষমতা দেয়।
- প্রতিটি Neuron দুটি ধাপে কাজ করে — প্রথমে $z = wx + b$ (Linear), তারপর $a = f(z)$ (Non-linear)।
- **Forward Propagation** মানে Input থেকে ধাপে ধাপে হিসাব করে Output-এ পৌঁছানো: $X \rightarrow z[1] \rightarrow a[1] \rightarrow z[2] \rightarrow \hat{y}$
- Matrix Notation শুধু একই ধরনের অনেকগুলো হিসাবকে সংক্ষেপে লেখার একটি পদ্ধতি — এতে নতুন কোনো ধারণা নেই।
- **Universal Approximation Theorem** MLP-এর তাত্ত্বিক ক্ষমতা ব্যাখ্যা করে, কিন্তু এর মানে এই নয় যে MLP সবকিছু সমাধান করতে পারবে।

14. Quick Revision

Core Flow



চিত্র 7: Forward Propagation-এর মূল প্রবাহ — সংক্ষিপ্ত রিভিশনের জন্য।

Core Equations

$$z^{[1]} = W^{[1]}X + b^{[1]}$$

$$a^{[1]} = f(z^{[1]})$$

$$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}$$

$$\hat{y} = f(z^{[2]})$$

মনে রাখার মতো সবচেয়ে গুরুত্বপূর্ণ ধারণা

- SLP = কোনো Hidden Layer নেই → শুধু Linear সমস্যা সমাধান করতে পারে।
- MLP = Hidden Layer আছে → Non-linear সমস্যা (XOR সহ) সমাধান করতে পারে।
- z = Activation-এর আগে, a = Activation-এর পরে — এই দুটি কখনো গুলিয়ে ফেলা যাবে না।
- Forward Propagation একটি সরল ধারাবাহিক প্রক্রিয়া: Weighted Sum → Activation → পরের স্তরে পাঠানো — এটাই বারবার ঘটে।
- Matrix Notation ভয় পাওয়ার কিছু নেই — এটা শুধু ছোট ছোট হিসাবগুলোকে একসাথে কমপ্যাক্টভাবে লেখার একটি উপায়।

Key Takeaway

একটি $3 \rightarrow 2 \rightarrow 1$ নেটওয়ার্কের দিকে তাকিয়ে এখন আপনি নিজের ভাষায় ব্যাখ্যা করতে পারবেন — Input কীভাবে Hidden Layer-এ যায়, z কীভাবে তৈরি হয়, Activation কীভাবে a তৈরি করে, Hidden-এর Output কীভাবে Output Layer-এ যায়, এবং শেষ পর্যন্ত কীভাবে $\hat{y}$ পাওয়া যায়।